

APENDEX FOR RASPBERRY PI :

```
import cv2

from ultralytics import YOLO

import threading

import time

import RPi.GPIO as GPIO

# GPIO SETUP

GPIO.setmode(GPIO.BCM)

FPGA_FRONT = 22 # From FPGA (LED1)

FPGA_BACK = 23 # From FPGA (LED2)

GPIO.setup(FPGA_FRONT, GPIO.IN, pull_up_down=GPIO.PUD_DOWN)

GPIO.setup(FPGA_BACK, GPIO.IN, pull_up_down=GPIO.PUD_DOWN)

# Output pins (Pi -> FPGA or motor control)

OUT_FRONT = 17

OUT_LEFT = 27

OUT_RIGHT = 24

GPIO.setup(OUT_FRONT, GPIO.OUT)

GPIO.setup(OUT_LEFT, GPIO.OUT)

GPIO.setup(OUT_RIGHT, GPIO.OUT)

# YOLO SETUP

model = YOLO("prabha.pt")

model.to("cpu")

#

frame = None

running = True

lock = threading.Lock()

# CAMERA THREAD

THREAD CONTROL
```

```

def capture_frames():
    global frame, running
    cap = cv2.VideoCapture(0)
    cap.set(cv2.CAP_PROP_FRAME_WIDTH, 640)
    cap.set(cv2.CAP_PROP_FRAME_HEIGHT, 480)
    cap.set(cv2.CAP_PROP_BUFFERSIZE, 1)
    while running:
        ret, img = cap.read()
        if ret:
            with lock:
                frame = img.copy()
            cap.release()
            threading.Thread(target=capture_frames, daemon=True).start()
        frame_count = 0
    # MAIN LOOP
    try:
        while True:
            # ---- Read FPGA Sensors ----
            fpga_front = GPIO.input(FPGA_FRONT)
            fpga_back = GPIO.input(FPGA_BACK)
            # ---- Get Frame Safely ----
            with lock:
                if frame is None:
                    continue
                img = frame.copy()
                start = time.time()
                frame_count += 1
            # Skip frames (performance)
            if frame_count % 2 != 0:

```

```

continue

# ---- YOLO Detection ----

results = model(img, imgsz=256, conf=0.6, verbose=False)

h, w, _ = img.shape
left_detect = 0
right_detect = 0
center_detect = 0
annotated = img.copy()

for box in results[0].boxes:
    conf = float(box.conf[0])
    cls = int(box.cls[0])
    label = model.names[cls]
    if conf > 0.75 and label in ["boat", "person", "ship"]:
        x1, y1, x2, y2 = map(int, box.xyxy[0])
        center_x = (x1 + x2) // 2

        # Draw box
        cv2.rectangle(annotated, (x1, y1), (x2, y2), (0,255,0), 3)
        cv2.putText(annotated, f"{label} {conf:.2f}",
            (x1, y1-10),
            cv2.FONT_HERSHEY_SIMPLEX,
            0.7, (0,255,0), 2)

        # Zone detection
        if center_x < w//3:
            left_detect = 1
        elif center_x > 2*w//3:
            right_detect = 1
        else:
            center_detect = 1

```

```
# DECISION LOGIC

# Priority: YOLO > FPGA

if center_detect:
    decision = "STOP (Vision)"
    GPIO.output(OUT_FRONT, 0)
    GPIO.output(OUT_LEFT, 0)
    GPIO.output(OUT_RIGHT, 0)
elif left_detect:
    decision = "TURN RIGHT (Vision)"
    GPIO.output(OUT_FRONT, 1)
    GPIO.output(OUT_LEFT, 0)
    GPIO.output(OUT_RIGHT, 1)
elif right_detect:
    decision = "TURN LEFT (Vision)"
    GPIO.output(OUT_FRONT, 1)
    GPIO.output(OUT_LEFT, 1)
    GPIO.output(OUT_RIGHT, 0)
elif fpga_front == 1:
    decision = "STOP (Ultrasonic Front)"
    GPIO.output(OUT_FRONT, 0)
    GPIO.output(OUT_LEFT, 0)
    GPIO.output(OUT_RIGHT, 0)

elif fpga_back == 1:
    decision = "BACK OBSTACLE (Warning)"
    GPIO.output(OUT_FRONT, 1)
else:
    decision = "MOVE FORWARD"
    GPIO.output(OUT_FRONT, 1)
```

```

GPIO.output(OUT_LEFT, 0)

GPIO.output(OUT_RIGHT, 0)

# FPS

fps = 1 / (time.time() - start)

cv2.putText(annotated, f"{decision}",
(10, 60),
cv2.FONT_HERSHEY_SIMPLEX,
0.8, (0,0,255), 2)

cv2.putText(annotated, f"FPS: {int(fps)}",
(10, 30),
cv2.FONT_HERSHEY_SIMPLEX,
1, (255,0,0), 2)

# Debug print
print(f"FPGA Front={fpga_front}, Back={fpga_back} | Decision={decision}")

cv2.imshow("Autonomous System", annotated)

if cv2.waitKey(1) & 0xFF == ord('q'):
    running = False
    break

# CLEANUP

except KeyboardInterrupt:
    pass

finally:
    GPIO.cleanup()
    cv2.destroyAllWindows()

```